

ChartLibre

User Manual

Application version 0.1.0

A desktop application for exploring data and building scientific charts

Contents

1	Introduction	3
1.1	Who this manual is for	3
1.2	Key concepts	3
2	Starting up and managing databases	4
2.1	Saving and duplicating a project	4
3	The main window	5
3.1	Navigation rail (left)	5
3.2	Data panel	6
3.2.1	Editing a table by hand	6
3.3	Chart panel	6
3.3.1	Right-click menu	7
3.3.2	Measuring, annotating and selecting points	7
4	Importing data	9
4.1	Exporting data	10
5	Query Builder	11
6	Creating a chart	12
6.1	Chart types	12
6.1.1	Pairwise data (x, y)	12
6.1.2	Statistical distributions	12
6.1.3	Gridded data	13
6.1.4	Irregularly gridded data	13
6.1.5	3D and volumetric data	13
6.2	How a renderer reads a series	13
6.3	Adding more series or axes	14
6.4	Customizing a chart	14
7	Series operations	16
7.1	Operating on a surface	19
7.2	Testing hypotheses: Statistics	20
7.3	Moving a series: Geometry	20
8	Appearance and styles	22
8.1	Matplotlib styles	22
8.2	Application settings	22
9	Application log	23
10	Demo projects	24
11	Credits	25
12	Advanced: extending ChartLibre	26
12.1	Writing a custom chart renderer	26
12.2	Writing a custom series operation	27

Introduction

ChartLibre is a desktop application for Windows, macOS and Linux that lets you import tabular data, query it with SQL, and turn it into scientific charts — from histograms to 3D surfaces — without writing code.

Every project is a single file with the extension `.dhub`: a SQLite database that holds both the imported data and the definitions of every chart (figures, axes, series and their drawing options). The file is therefore self-contained and portable — moving or sharing it carries both the data and every visualization built on top of it.

Who this manual is for

This manual describes the application as it presents itself to a user: the main window, importing data, building queries, creating and customizing charts, running statistical operations on series, and the application's settings. A final chapter, Section 12, is aimed at developers who want to extend ChartLibre with a new chart type or a new analysis operation.

Key concepts

- **Database (`.dhub`)**: the project file. Holds data tables and figures.
- **Table**: a set of data, either imported (from CSV, Excel, ...) or produced by a saved query.
- **Saved query**: a named SQL statement, usable anywhere a table is.
- **Figure**: one tab in the chart panel; can hold one or more axes.
- **Axis**: a single chart inside a figure, with a chart type (histogram, scatter, ...) and one or more series.
- **Series**: the data drawn on an axis, defined by a SQL query that produces the columns (**roles**) the chart type requires (e.g. `x`, `y`).
- **Renderer**: the piece of code behind a chart type — it turns a series' data into a Matplotlib drawing. See Section 6.1.

Starting up and managing databases

On first launch, or whenever no file is given, ChartLibre asks which database to open:

New creates an empty `.dhub` database at a path you choose.

Open opens an existing `.dhub` file.

Load demo loads one of the shipped demo projects and opens it immediately — useful for exploring the application without your own data.

The last database opened is remembered and offered again on the next launch.

Saving and duplicating a project

A SQLite database writes its own changes straight to disk on every operation, so there is no separate “Save” for data. Two related commands are available instead:

Save As... saves the current database under a new name/path and switches to working on that copy, leaving the original untouched.

Optimize DB checks the database for problems, reports them, and compacts it (`VACUUM`, `ANALYZE`) to shrink it on disk and speed up opening it.

Database Info the current project’s file path and size, everything SQLite itself can report about the file (table/row totals, page accounting and how much **Optimize DB** would reclaim, encoding, journal mode, SQLite’s own version, and the file’s creation/modification time from the filesystem), a list of every table with its row count and which import (if any) it is linked to, and buttons to export a table (CSV/Excel) or refresh a linked one on the spot.

The main window

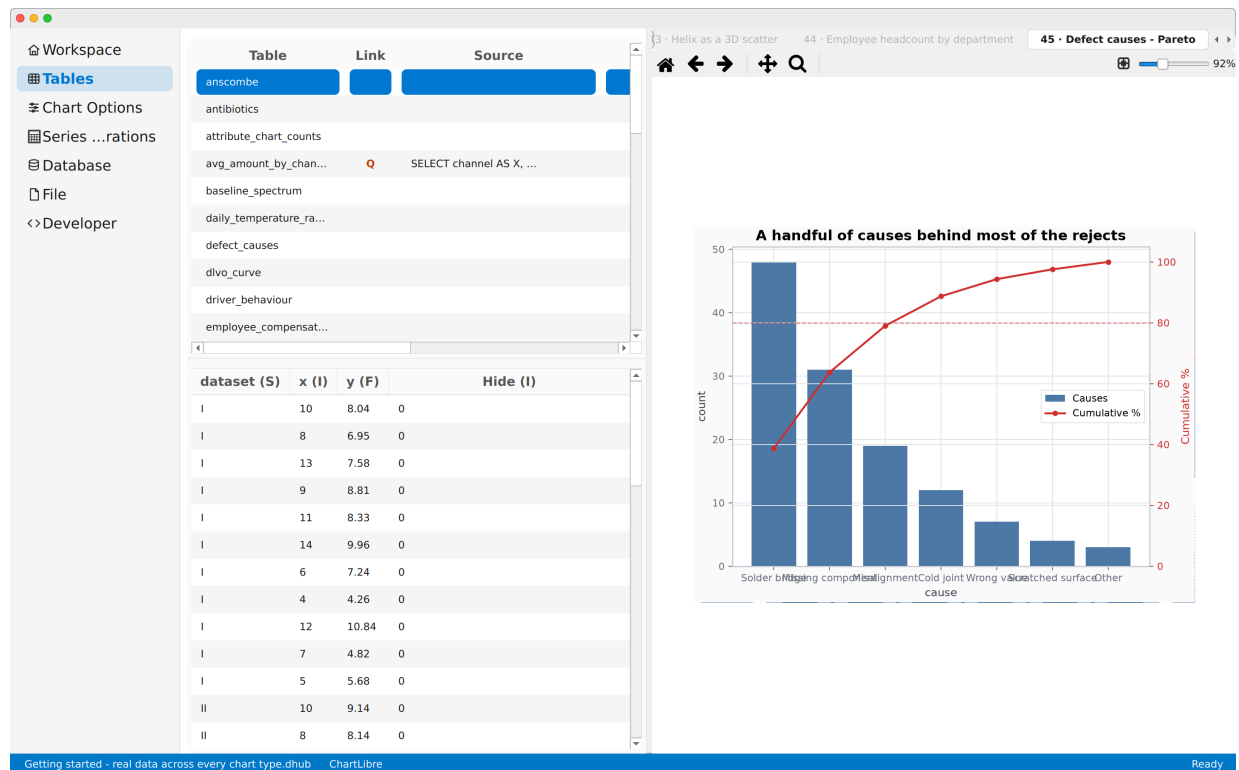


Figure 1: The main window on the Tables page: the navigation rail, the table list and the data preview on the left, the chart panel on the right.

The figures in this manual are taken in the **macOS native** style, which is what the application wears on a Mac. On Windows and Linux the same screens wear the **Fluent** style instead: the panels, the pages and every control are the same and in the same place, but the navigation rail is a column of square tiles rather than a sidebar of rows, and the window carries its own title bar rather than the traffic lights. Regenerate the figures with `python docs/manual/make_screenshots.py`.

The main window splits into two areas.

Navigation rail (left)

A vertical column of icons switches between the application's main sections:

- **Workspace** — collapses the panel beside the rail down to the rail's own width, for when the chart panel needs the room; clicking it again restores whatever section was open.
- **Tables** — the list of tables and queries in the database, with a data preview.
- **Chart Options** — properties of the currently selected figure, axis and series.
- **Series Operations** — **Plot**, plus the analysis tools (fit, statistics, filtering, ...) that apply to a series' data.
- **Database** — the Query Builder (see Section 6) and an overview of the database itself: its path and size, every table's row count, and per-table CSV/Excel export and import-link refresh.
- **File** — New, Open, Import and Load demo; Save and Save As; and the Open Recent list.
- **Developer** — scaffolding tools: the translation catalogue editor, the series-operation builder, the custom function creator, and the renderer helper (see Section 12).

Off macOS the rail ends with two more tiles, **Settings** and **Help**, which open the same things the macOS menu bar carries in its application and Help menus.

Data panel

Lists the tables in the database (saved queries are marked with a **Q** icon) and, once one is selected, shows a preview with its first rows and columns. This is also where importing data and opening the Query Builder start.

Its **Source** column says where each row’s data actually comes from: a filename for a file or web import, “connection → table” for a database import, the query itself (truncated, with the full text in the tooltip) for a saved query, and nothing for a table with no link. Right-clicking a saved query offers **Edit...**, which reopens it in the Query Builder.

Editing a table by hand

The preview is read-only: it is there to show what a table holds while a chart is built from it. To change the data itself, right-click the preview and choose **Edit table...**

Cells	Double-click one and type. An emptied cell becomes NULL rather than an empty string, which in a numeric column is the difference between “not measured” and a value that quietly turns the column into text.
Rows	Add row appends an empty one; Insert row above puts one before the selected row; Delete rows removes every row with a selected cell. The toolbar is icons rather than labelled buttons — hover one for its name — so that the dialog fits a laptop screen with the table, rather than the buttons, taking the room.
Columns	Type a name, pick a type, then Add column (at the end) or Insert before selected . Rename selected renames the column the cursor is in, and Delete selected removes it and its contents.
Managed columns	Hide marks rows every chart skips; ClusterId is what the Clustering operation writes into. The buttons here create, reset and invert them. These used to live in the preview’s own right-click menu, several levels down a menu otherwise about looking rather than changing.

Every change is written to the database as it is made — a SQLite table has nowhere to be “saved” to — and yet **Cancel** still puts the table back. Both are true because of the undo snapshot the editor takes before its first change: **OK** leaves that snapshot in the history like any other action, so the session can still be undone afterwards, and **Cancel** restores it there and then. Restoring is not itself undoable, so **Cancel** asks first.

Inserting a row **above** another renumbers the rows below it, and inserting a column **before** another rebuilds the table in the new order. Both keep the data; neither is offered on a table whose own rowid is one of its columns (an **INTEGER PRIMARY KEY**), because renumbering would be rewriting the key values themselves — there, add the row at the end instead.

Chart panel

Occupies the right side of the window and is organized into tabs — each tab is a **figure**. The toolbar above the chart offers Matplotlib’s usual navigation tools:

Home	Return the chart to its initial view
Arrows $\leftarrow/\rightarrow$	Step back/forward through the zoom history
Pan	Drag to move the visible area
Zoom	Draw a rectangle to zoom into an area

Zoom to fit brings the whole chart back within the panel's bounds in one click.

The **Plot** button, at the top of the Series Operations panel, creates a new figure, configured as described in Section 6.

Right-click menu

Right-clicking the chart opens a menu with, at the top, the actions below; clicking inside an axis (rather than on empty space) adds a second group naming the exact point clicked:

Reload	Redraw the chart from its saved definition, discarding any zoom/pan.
Copy	Copy the chart image to the clipboard.
Save	Save the chart as a picture, in the format chosen under Settings.
Export view as CSV...	Save the rows actually visible right now — after any zoom, and after downsampling — as a CSV file. Each row records which axis and series it belongs to.
Crosshair	A guide line under the pointer, shared across every axis of the figure (its horizontal half only ever appears on the axis actually being pointed at).
Link zoom/pan across axes	Zooming or panning one axis' x range applies the same range to every other axis of the figure — for comparing several time series stacked one above another.
Select points	Turns the pointer into a rectangle-selection tool — see below.
Live updates	Redraws the chart on a timer, for a source table a long-running import or an external process keeps filling in.
Delete	Removes this figure, after confirmation.

Clicking inside an axis additionally offers **Add vertical/horizontal line at ...** and **Add annotation here...**, and the ruler flow described next.

Measuring, annotating and selecting points

Measure from here / Measure to here, on the axis context menu, record a two-click measurement: the distance, Δx , Δy and the slope between the two points, kept on the chart like a reference line. Every measurement — like every annotation and reference line — has its own tab in **Overlay properties** (reachable from Chart Options), where it can be edited by exact numbers or deleted.

An annotation can also be repositioned directly: click and drag its text on the chart to move it, rather than opening Overlay properties to change its coordinates by hand.

Turning on **Select points** lets you drag a rectangle over the chart; every plotted point it covers, from every series on that axis, is picked up. Right-clicking while a selection is active offers two more actions:

Hide N selected point(s)	Removes those points from the chart without touching the underlying table — it simply narrows the series' own query. Undo brings them back.
New series from selection	Adds a new series, on the same axis, holding only the selected points — a quick way to isolate one region of a series (a peak, an outlier cluster) for its own analysis.

Importing data

Import opens a dialog with four ways to bring data in:

Open	A local file: <code>.csv</code> , <code>.tsv</code> , <code>.txt</code> , <code>.xlsx</code> , <code>.xslm</code> , <code>.xls</code> , <code>.json</code> , <code>.xml</code> .
Paste	Whatever tabular text is currently on the clipboard.
Database	One table — or the result of a query — read out of another database: SQLite (including another <code>.dhub</code> file), PostgreSQL, or MySQL.
Web	Whatever an <code>http(s)</code> URL returns — a CSV export, a JSON API, a spreadsheet.

Whichever way the data arrives, the same preview, column-type mapping and destination-table name apply before anything is written. For Excel sources with more than one sheet, the dialog asks which one to import.

The dialog's left panel is grouped into **Source** (where the data comes from, including the web row), **Read options** (destination table name, header row, skipped rows, delimiter, encoding) and **Columns** (the per-column type mapping). The read options describe how to parse **text**: when the source is a database table they are disabled, because a table already has its own column types, no delimiter and no header row to detect.

Database opens its own small window:

1. Pick an engine — a SQLite file, another ChartLibre project (`.dhub`), PostgreSQL or MySQL.
2. Fill in a file path, or a host/port/username/password. For a server, **Connect** asks it which databases exist and offers them in a list, so the name does not have to be typed from memory; picking one lists its tables.
3. Choose a table — never any internal bookkeeping tables ChartLibre itself may have added, for a SQLite/`.dhub` source — or tick **Use a query** and write a `SELECT` instead, for a join, a filter or an aggregate.
4. Confirm to bring the result back to the import dialog like any other source.

The last connection is remembered, password excepted, so reopening the window lands on the same server and table.

Web has its own row: a quick-pick menu of ready-made public datasets grouped by subject, the URL itself, and **Fetch**. Picking an entry fills the URL in rather than downloading immediately, so it can be read and edited first. **Add source** saves a URL of your own to that menu under a name you choose, and **Delete source** removes one you added — the entries that ship with the application cannot be deleted.

An imported table stays available to every query and chart in the project: importing is a one-time read, not a live link back to the original file, database or URL — the data does not change on its own afterwards. Every source **except** a paste can be re-read later, though: right-click the table and choose **Update link** to replace its contents with a fresh read from the same file, database table, query or URL.

A PostgreSQL or MySQL connection's password is never saved — not in the project file, not anywhere on disk. **Update link** asks for it again each time, because a `.dhub` project is exactly

the kind of file that gets copied, emailed or committed without a second thought, and a password sitting in plain JSON inside it would travel right along.

Web only fetches `http://` and `https://` URLs — never a local path — so pasting a link here cannot read a file off disk by way of a `file://` address.

Exporting data

The content of a table or the result of a query can be exported with:

Export CSV writes the data to a `.csv` text file.

Export Excel writes the data to an `.xlsx` workbook.

Query Builder

The **Query Builder** (menu **Query Builder**) writes, checks and saves SQL statements that can be used as tables — handy for filtering, joining or aggregating imported data without duplicating it.

The editor has a **Run** button to check the result before saving, and a set of ready-made snippets that insert the right SQL skeleton:

Select	SELECT over a chosen table
Filter	WHERE with the comparisons spelled out
Join	LEFT JOIN keeping every row of the chosen table
Order	ORDER BY with an explicit direction
Summary	Count, average, min and max, grouped by a column
Union	UNION ALL of two tables with the same columns

A saved query appears in the table list with a **Q** icon and can be used as the data source for any series, exactly like an imported table. Unlike a table, its content is computed on the fly rather than stored: if the source data changes, the query returns fresh results the next time it runs.

Creating a chart

Plot, at the top of the Series Operations panel, opens the chart-creation window: pick a **chart type** (a renderer) and define the first series — the SQL query that supplies the data.

Every chart type requires the query to produce columns with specific names, its **roles** — for instance **x** and **y** for a scatter plot. The window shows the roles a type requires and a link to that type’s Matplotlib documentation.

Chart types

ChartLibre ships 34 chart types (**renderers**), grouped into the same categories Matplotlib’s own documentation uses. Each is a self-contained piece of code that receives the rows a series’ SQL query returns and draws them; see Section 12.1 for how a new one is added.

The chart picker lists them section by section and its search box matches a name or a whole category, so typing “stat” brings up the statistical family at once.

Pairwise data (x, y)

Bar Chart	Vertical bar chart
Horizontal Bar Chart	Horizontal bar chart
Broken Bar	Interval bars grouped by category (Gantt-style)
Broken Bar (Vertical)	The same, stacked in columns
Scatter Plot	Scatter plot; optional color and size columns drive a colour map and marker sizing
Fill Between	The region between two y curves over a shared x — confidence bands, tolerance limits
Stack Plot	Series stacked into filled bands, showing how a total divides into its parts
Stem Plot	A stem from a baseline to each value — impulses, spectra, anything sampled at discrete x
Table	A data table, rows and columns of text rather than a plot
Text	Text labels at data points, spread apart to avoid overlap
Time Series	Time series, with numeric or timestamp x
Timeline	Dated events on a baseline, each on its own stem — release histories, event lists

Statistical distributions

Histogram	Histogram, supports multiple datasets
Box Plot	Box-and-whisker plot
Violin Plot	Estimated distribution of one or more samples
ECDF	Empirical cumulative distribution function
Stairs	A step outline over bin edges, for values that hold constant between them

Pareto Chart	Categories sorted by descending magnitude with a cumulative-percentage line
Pie Chart	Pie or donut chart of one series
Hexbin	2D density of x/y pairs binned into hexagons, for a scatter with too many points to read individually
Event Plot	A row of tick marks per series at each x where an event happened — a digital signal's edges, an error log, a spike train

Gridded data

Values sampled on a complete, evenly spaced x/y grid.

Contour Plot	Filled or line contours of z over a regular x/y grid
Heatmap	Raw per-cell colour over a regular x/y grid, no interpolation between cells
Quiver	An arrow per sample showing a vector field's direction and magnitude (u/v components)
Stream Plot	Streamlines traced through a vector field — where a flow goes, rather than what it does at each sample
Wind Barbs	A barb per point encoding speed in flags, readable where a field of arrows is not

Irregularly gridded data

The same quantities measured wherever they could be measured, with no grid to assume.

Contour Plot (Scattered)	Filled or line contours for scattered x/y/z data
Triangular Mesh	The triangulation itself, as edges and vertices — shows what an interpolating chart will assume between samples
Triangular Color Mesh	Scattered samples triangulated and filled by value

3D and volumetric data

Surface Plot	3D surface over a regular x/y grid
Surface Plot (Scattered)	3D triangulated surface for scattered (non-gridded) x/y/z data
Scatter Plot (3D)	Points in three dimensions, optionally coloured and sized by further columns
3D Line Plot	A trajectory or parametric curve through three dimensions, points connected in row order
3D Bar Chart	Bars rising from the x/y plane to z — categorical data in three dimensions

A chart type that needs a complete grid says so and draws nothing when the rows it is given are not one — it never interpolates the missing cells and calls the result a measurement. Its scattered counterpart is named in the log entry; switch the axis to that one instead.

How a renderer reads a series

A renderer never sees your raw table — it sees the result of the series' SQL query, aliased to its required **roles**. A Scatter Plot needs x and y, so its series query is written as:

```
SELECT pressure AS x, flow AS y FROM operating_points
```

Optional roles work the same way and unlock extra behaviour when present — for instance `color` and `size` on a Scatter Plot drive a colour map and marker sizing, and `yerr/yerr_low/yerr_high` add error bars. The chart-creation window and the Chart Options panel both list which roles a given type accepts.

Every renderer also splits its adjustable settings into two families, both editable from Chart Options:

- **Plot arguments** (forwarded to Matplotlib as-is — line width, marker, transparency, colormap, ...).
- **Options** (consumed by the renderer itself and never forwarded to Matplotlib — whether to draw a legend, how many bins, which fit to overlay, ...).

Adding more series or axes

A figure can hold several axes (each drawn side by side or overlaid within the same tab), and each axis can hold several series, up to the maximum the chart type allows — some types, like Pie Chart, accept only one series; the application then proposes a new axis instead of a second series that would be dropped.

Add series in the Chart Options panel adds a new series to the selected axis.

Customizing a chart

From Chart Options, depending on the selected level:

- **Figure:** title, layout of its axes.
- **Axis:** title, axis labels, grid, legend, and the chart type's own options (e.g. the number of bins for a histogram).
- **Series:** the source query, colour, line or marker style, and any Matplotlib arguments forwarded straight to the drawing call (line width, transparency, ...).

Every setting shows a contextual description, so reading Matplotlib's own documentation is rarely necessary to understand what a parameter does.

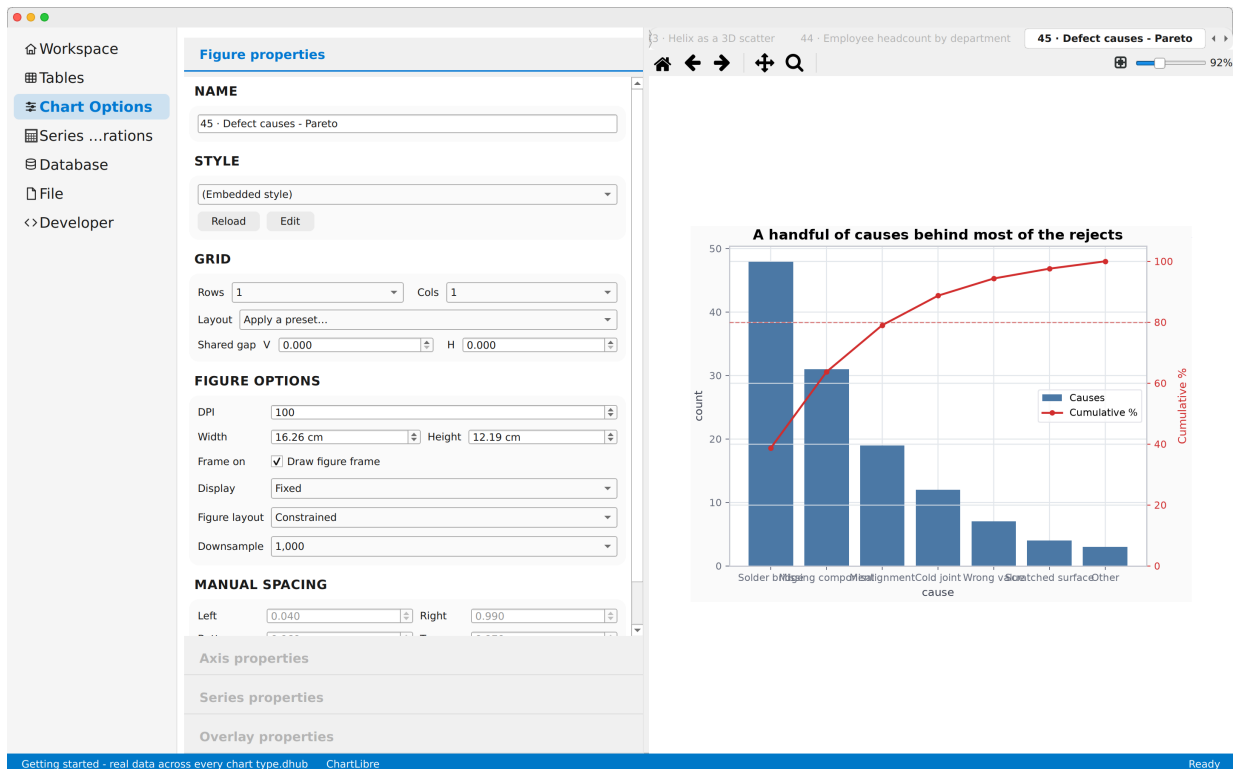


Figure 2: Chart Options: the figure, axis, series and overlay property panels, one collapsible section each, over the chart they apply to.

Overlay properties, also in Chart Options, lists an axis' annotations, reference lines and measurements — one tab each — as editable tables: exact position, text, colour and every other drawing option by value, rather than only by dragging on the chart.

Series operations

Series Operations applies statistical or mathematical transformations to an existing series' data, previewing the result before it is committed to the chart. Every operation dialog shares the same layout: on the left, the source axis/series, a model and its parameters; on the right, a preview of the result and a log of the computation.

The panel itself is a grid of square buttons grouped by what they are for — **Plot**, **Analysis**, **Statistics**, **Signal Processing**, **Modeling** — with a bar along the bottom that describes whichever button the pointer (or the keyboard focus) is on.

A calculation that can take a while — a **Fit**, above all — runs in the background: the window stays usable, **Preview** and **OK** are switched off while it runs, and a **Stop** button appears beside **Preview**. **Stop** ends the calculation at its next step and changes nothing; closing the window stops it too.

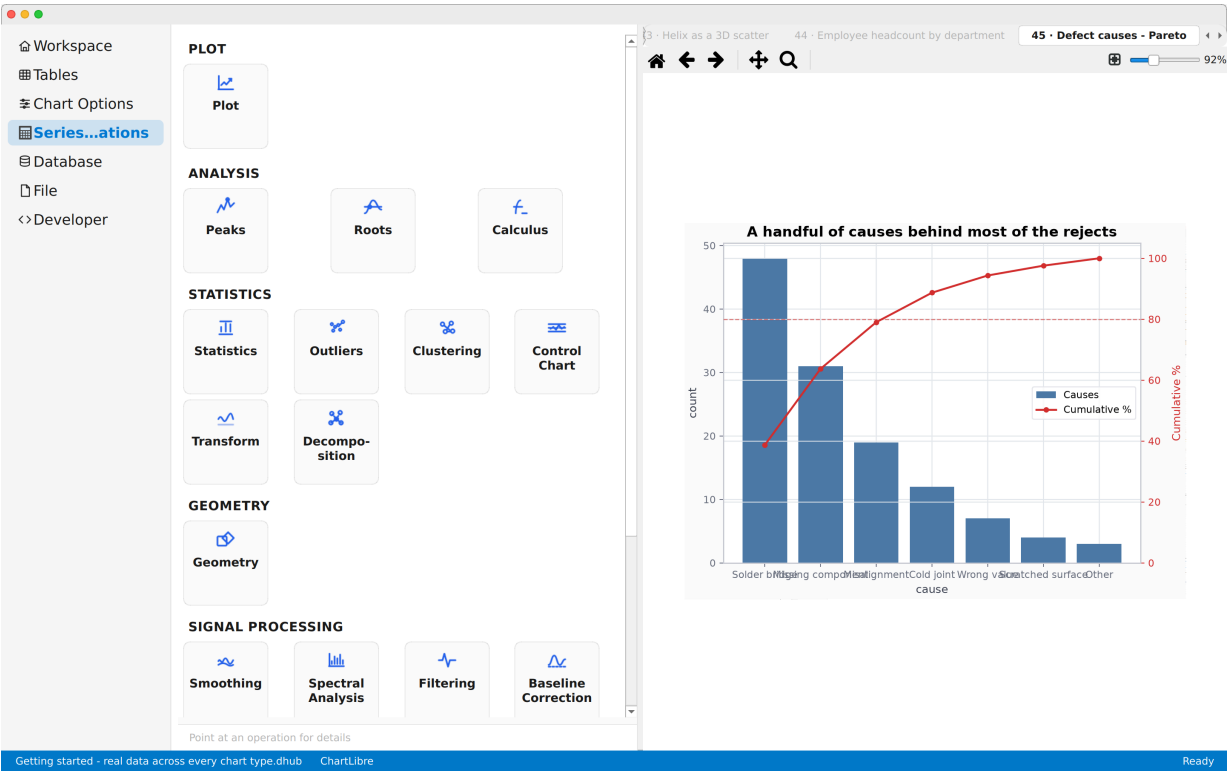


Figure 3: The Series Operations page, with the hint bar along the bottom.

Peaks	Find and measure peaks	Detects peaks in a series and reports their position, height and width.
Roots	Find where a series crosses a level	Locates the x values at which a series crosses a chosen level — zero by default — by interpolating between the samples either side.
Calculus	Differentiate or integrate	Computes the numerical derivative or the cumulative integral of a series, with smoothing built into the first and baseline subtraction into the second. On a dated x axis, the result is scaled against the

		sample spacing's own natural unit (day, hour, ...) rather than raw seconds, so “dy/dx” reads as “per day” for daily data instead of a tiny per-second number.
Statistics	Compute metrics	Reports descriptive statistics and hypothesis tests for the checked series: one-sample, normality, paired and independent-sample tests, a comparison of all of them as groups (ANOVA, Welch's ANOVA, Kruskal-Wallis, Tukey), correlation and distribution fitting. See Section 7.2.
Outliers	Detect anomalies	Flags points that deviate from the rest of a series, by a chosen statistical criterion (Z-score, IQR, ...) or a shape-aware model (Isolation Forest, Local Outlier Factor, One-Class SVM, Elliptic Envelope) that judges a point by where it sits relative to its neighbours rather than by its value alone.
Clustering	Group similar data	Groups a series' points into clusters (k-means, DBSCAN, ...) and labels each point by cluster.
Regression	Robust and ML regression	Fits a curve through noisy or outlier-heavy data with a model that does not assume a known shape: RANSAC and Huber (robust linear, tolerant of gross outliers), Isotonic (guaranteed monotone), or Random Forest / Gradient Boosting (flexible, nonparametric).
GP Regression	Fit with an uncertainty band	Gaussian process regression: a smooth fitted curve plus a $\pm 2\sigma$ credible band as two more series, so the result says how sure it is, not only what it thinks the value is.
Transform	Rescale a distribution	Reshapes a series' values with a Power, Quantile, Standard or Robust transform — the usual prerequisite for a method (a curve fit, a control chart) that assumes roughly normal or well-scaled data.
Decomposition	Combine several series	Projects several selected series, resampled onto a shared grid, into fewer dimensions: PCA/ICA/NMF for an explained-variance-ranked decomposition, or t-SNE/Isomap/Locally Linear Embedding for an exploratory 2D embedding of their shared structure. The one operation here that

		reads several series together rather than one at a time.
Control Chart	Monitor process stability	Builds a statistical control chart and flags rule violations — I-MR, X-bar-R and X-bar-S for measurements, p, np, c and u for counts.
Smoothing	Reduce noise	Applies a smoothing model (moving average, Savitzky-Golay, ...) to reduce noise while preserving the underlying shape.
Spectral Analysis	Analyse frequencies	Power spectral density, cross-spectral density, coherence, magnitude/phase spectra and auto/cross-correlation, on their own new axis.
Filtering	Filter, detrend or demodulate	Low/high/band-pass and band-stop filtering (Butterworth, Chebyshev, Bessel, FIR), trend removal, and the Hilbert envelope of a signal.
Baseline Correction	Subtract a background	Removes a drifting background from a spectrum — asymmetric least squares or a rubber band — and keeps the baseline it removed as its own series.
Fit	Fit data models	Fits a mathematical model (Gaussian, exponential, polynomial, a user function, ...) to a series by least squares, and adds the fitted curve alongside the data. Reports each parameter with its standard error, t, p and 95% confidence interval, can shade the 95% confidence and prediction bands around the curve, and optionally adds a residuals chart and a measured-vs-fit chart.
Interpolation	Fill missing values	Fills gaps in a series (linear, spline, nearest, ...), producing a complete curve from a sparse one.
Function	Plot a function	Evaluates a function over a range and plots it — the one operation that reads no source series at all.
Geometry	Move a series' coordinates	Rotates, translates, roto-translates, scales, mirrors or shears a series — in the plane, or in space for a series with a z column. Alone among these, it computes nothing and stores nothing: the result is the source series' own query wrapped in the affine expressions, so the moved series follows its source rather than being a copy of it. See Section 7.3.

A typical workflow is: select the axis and series to operate on, choose a model and its parameters, click **Preview** to see the result superimposed on the chart, adjust the parameters if needed, then confirm to add the result as a new series (or table) permanently. See Section 12.2 for how a new operation is added.

Nothing is computed while you are editing. Changing a model, a parameter or the selected series marks the current result stale and leaves it at that; only **Preview** and **OK** actually run the computation. So a dialog that costs a second or two to evaluate can be set up in peace, and a half-typed parameter is never computed against.

An operation reads **one** source series. If several are ticked, the first one is used and the others are ignored — the status bar says which one it took. To operate on a different series, untick the others, or move the one you want to the top of the list. **Decomposition** and **Statistics** are the exceptions: Decomposition is built to read several series together, and Statistics reports on every ticked series - and compares them, pair by pair or as groups.

Operating on a surface

Four of the operations above also understand a series that carries a z role — a surface ($z = f(x, y)$) — rather than a plain x/y curve. They read the same series and the same parameters; what changes is what the question means in three dimensions.

The **Axis / Series** panel captions whichever series is selected with the shape it found, so this is visible before anything is pressed:

2D (x, y)	An ordinary curve — every operation's usual case.
3D on a grid (x, y, z)	The rows form a complete, regular x/y grid, so the surface is read exactly as measured.
3D scattered (x, y, z)	Points with no grid behind them. The surface is interpolated onto one, and stays undefined (blank) outside the samples' convex hull rather than being extrapolated into territory nothing was measured in.
vector field (x, y, u, v)	A u/v field. Informational only for now — no operation computes on it.

What each of the four does with a surface:

Fit	Fits a surface model by least squares, the same way it fits a curve. Five are built in — Plane ($z = a + b x + c y$), Paraboloid (an elliptic bowl), Gaussian2D (a bump or dip on a flat baseline), Saddle (rises along x , falls along y) and Ripple2D (a separable oscillation) — listed under their own Surfaces category beside the 1D models, with User surfaces for your own.
Function	Evaluates a surface function over a grid it generates, with no source series to read — the 3D counterpart of plotting $y = f(x)$ over a range.
Peaks	Finds local maxima and minima on the surface rather than along a curve. The same prominence and threshold parameters apply, so moving from a curve to a surface does not also mean learning new ones.

- Roots** Traces the **level curve** $z = \text{level}$, not a list of crossing points: a surface's level set is generally a curve, and a saddle's $z = 0$ set is two crossing lines rather than two numbers.
- Calculus** Reports the surface's gradient — its magnitude at each sample, with the dz/dx and dz/dy components carried alongside — and the volume under it, in place of the derivative and the integral of a curve.

Testing hypotheses: Statistics

Statistics reports on the series checked in its list; it draws a chart only if asked (**Add a chart**). The **Model** chooses what is reported:

Descriptive statistics	Mean, trimmed, geometric and harmonic means, median, mode, spread (standard deviation, IQR, MAD), shape (skewness, kurtosis) and the 95% confidence interval of the mean.
One-sample tests	Is the centre of a series equal to a reference value? t-test, Wilcoxon signed-rank and sign test.
Normality / shape tests	Shapiro-Wilk, D'Agostino, Anderson-Darling, Jarque-Bera and more.
Paired-sample tests	Two series measured on the same units (before/after on the same parts): paired t, Wilcoxon, sign test — for every pair of checked series.
Independent-sample tests	Two groups of different units: Welch's t (the default reading: it does not assume equal variances), Student's t, Mann-Whitney U, Levene's test for equal variances, and Cohen's d as the size of the difference — for every pair.
Group comparison (ANOVA)	All checked series as groups: one-way ANOVA with η^2 , Welch's ANOVA, Kruskal-Wallis with ϵ^2 , Levene's test, and Tukey's HSD for every pair with its adjusted p-value and 95% interval. Where Levene says the variances differ, read Welch's ANOVA.
Correlation / association	Pearson, Spearman, Kendall and the regression slope, for every pair.
Distribution fit	Ranks candidate distributions for each series by AIC, BIC or the KS statistic.

The results have been checked against published reference values: SciPy's documented ANOVA example and, for the fit, the certified NIST StRD data sets.

Moving a series: Geometry

Geometry answers “put this where that is”: rotate a scan onto a reference frame, shift a baseline, flip a profile, scale a model onto measured units. Pick a transform, set its numbers, and **Preview** draws where the points were and where they went.

- Rotate** Turns the series about a centre, counter-clockwise, by an angle in degrees.
- Translate** Adds a fixed amount to every x and y (and z). The one transform with no centre — moving something does not leave any point of it where it was.

Roto-translation	A rotation about the centre followed by a translation: a rigid motion, the way a part is placed on a reference frame. Shape and size never change.
Scale	Stretches x and y (and z) by their own factors about the centre. Different factors change the shape, not only the size.
Mirror	Reflects across a horizontal, vertical or diagonal line through the centre.
Shear	Leans the series over: so much x added per unit of y, or the other way about.

In 3D. A series with a z column (a 3D scatter, a 3D line, a surface) is moved in space. Its rotation takes three angles — about x, then about y, then about z, each counter-clockwise looking down that axis (right-hand rule) — and translation, scale and centre gain a z value. **Mirror** and **Shear** act in the x-y plane and leave z as it is. The dialog shows only the values that apply to the selected series: one angle for a 2D series, three for a 3D one. The preview is then a 3D view, with the series' bounding box drawn as a cage before and after.

Centre on the data — on by default — uses the middle of the series' own extent as the fixed point, which is what “turn this shape” usually means. Turn it off to name a centre explicitly, and the origin is then just one choice of centre among others.

The results panel is a picture rather than a table, because a rotation is one: the source in blue, the result in red, and — with **Show the deformation grid** — a square mesh over the source's extent drawn before and after, which is what makes a shear or an uneven scale legible at a glance.

Nothing is computed, and no table is written. The series this creates carries a SQL query: the source series' query, wrapped in the transform's own arithmetic, with a 2D rotation spelled out as `cos(radians(30))` rather than as a decimal (a 3D rotation, whose coefficients mix three angles, is written with its numbers). Three things follow. The moved series **follows its source** — change the source query and the transform applies to whatever it now returns. A large series costs a query rather than a second copy of itself in the project file. And the transform can be read and adjusted afterwards in the Query Builder, which is where anyone wanting 30.5° instead of 30° will end up.

A surface operation never invents data. On a gridded series it works from the grid the rows actually form; on a scattered one it interpolates onto a grid and leaves everything outside the convex hull of the samples blank. The **3D Series Operations** demo (see Section 10) is built by calling these same methods, so it shows exactly what they produce.

Appearance and styles

Matplotlib styles

Edit Matplotlib styles lets you adjust the base graphical parameters (rcParams) that govern the look of every chart — default colours, fonts, line widths. From this window you can:

- Load an existing `.mplstyle` file into the editor.
- Add an rcParam property to the editor.
- Reset a property to its Matplotlib default.
- Save the changes as a new `.mplstyle` file.

Application settings

Settings gathers the general preferences:

App style the look of the interface controls. Includes the platform’s native styles plus, when installed, extra Qt styles (e.g. Breeze, Oxygen, QtCurve).

Language the interface language. **Auto** follows the operating system’s language and falls back to English when it names one with no catalogue. Every language that has a catalogue folder is then listed by its own name — English and **Italiano**, which is complete. A language whose catalogue is only started appears under its bare code (`fr`) and will still read mostly English; the **Edit Localization** editor (see Section 12) is where it gets filled in.

Neither setting is applied live, and the dialog says so: **the style and the language are applied the next time the app starts**. That is deliberate rather than unfinished — re-styling every widget in a running application restarts the Qt style underneath them all, which is fatal where the HTML results pane is a `QWebEngineView`, and a language switch would only reach menus and labels built after it, leaving the rest of the window in the old one.

Application log

The **Log viewer** shows the history of the application's internal operations (startup, opening a database, errors) — useful for diagnosing unexpected behaviour or attaching details to a bug report.

Demo projects

Load demo opens a pre-filled `.dhub` database with sample tables and a set of figures already configured across several chart types. Twenty-one ship with the application, chosen from a list that shows each one's summary as it is selected — between them they cover every chart family in Section 6.1 and give several of the series operations a dataset built to suit them (a sparse curve for **Interpolation**, a drifting spectrum for **Baseline Correction**, a three-tone signal for **Filtering** and **Spectral Analysis**, a Gaussian bump and a saddle for the surface operations). It is the fastest way to explore the application's features without preparing your own data, and a good place to copy a series' or an axis' settings from into a real project. It loads straight into your home directory under the demo's own name — no save dialog to answer first — and loading the same demo again simply replaces that file with a fresh copy, so an edited demo is never mistaken for your own work.

Credits

Credits lists the application, the version in use, and the open-source libraries it is built on (including Qt/PySide6, Matplotlib, NumPy, pandas, SciPy, statsmodels and scikit-learn).

Advanced: extending ChartLibre

ChartLibre discovers chart types and series operations the same way: by scanning a folder for Python classes that directly subclass a known base class, at import time — no registration list to edit and keep in sync. Dropping a well-formed file into the right folder is enough for it to appear in the application.

The **Developer** menu (macOS: its own menu; elsewhere: the Menu button’s Developer section) offers a GUI shortcut for the sections below, so writing the file by hand is the fallback, not the only way:

Renderer Helper a short form (name, category, description, link, required/optional roles) that scaffolds a new chart-type file.

Series Operation Builder the same, for a new series-operation dialog - the scaffolded file already runs, passing each selected series through unchanged, ready to have the real computation dropped in.

Function Creator the same, for a new fit function - the scaffolded `execute(x, p)` already evaluates a real polynomial in the declared parameters.

Edit Localization a table editor for a language’s translation catalogue, filterable to untranslated strings only, with a button to create a new language from scratch and an optional machine-translation pass to draft from.

Every scaffolded file from the first three goes under `user/` at the project root (`user/charts/`, `user/series_operations/`, `user/functions/`) rather than into `app/`’s own shipped source - kept separate on purpose, so a packaged build’s own files are never touched and a person’s own additions are never confused with what shipped.

Writing a custom chart renderer

A chart type lives entirely in one file under `app/charts/`, as a class that subclasses `BaseAxisRenderer` (from `app.charts.base`) **directly** — the discovery scanner in `app/scanners/axis_renderer_scanner.py` matches that exact base-class name in the source, even when the class also inherits behaviour from another renderer.

The class is described entirely through its own attributes:

Name	Display name, and the <code>chart_type</code> value stored in the database. Renaming it orphans existing axes unless an alias is added to <code>CHART_TYPE_ALIASES</code> in the scanner.
Category	Which family of plot this is, following Matplotlib’s own taxonomy (e.g. “Pairwise data”, “Statistical distributions”).
Description / Link	Shown in the chart picker and in the axis properties panel.
RequiredRoles / OptionalRoles	Column names the series’ SQL query must (or may) produce — see Section 6.1.
Kwargs	Matplotlib keyword arguments forwarded verbatim to the plot call, as <code>{name: metadata}</code> . The metadata (<code>default</code> , <code>type</code> , <code>min/max</code> , <code>kind</code> , <code>group</code> , <code>description</code>) drives the generated editor UI automatically.
Options	Settings the renderer consumes itself and never forwards to Matplotlib — same metadata shape, opposite destination.

MaxSeries	How many series this renderer can draw on one axis, or None for any number.
------------------	--

The class then implements one method:

```
def render_axis(self, ax, series: list[SeriesData], options: dict | None = None) -> None:
    ...
```

which receives one **SeriesData** per series — already queried, with its columns in **sd.df** — and draws them onto **ax** (a Matplotlib Axes).

sd.df is a **SeriesFrame** (**app.data.series_frame**): columns of NumPy arrays read straight from SQLite, read the way a DataFrame is (**sd.df["x"]**, "x" in **sd.df.columns**, **sd.df.loc[mask, "color"]**), with **sd.df.to_pandas()** available for the rare operation that genuinely needs a DataFrame. The developer guide's §4.1 covers it.

Practical steps:

1. Create **app/charts/my_chart.py**.
2. Import **BaseAxisRenderer** and, if useful, **SeriesData** from **app.charts.base**.
3. Define a class, e.g. **MyChartAxisRenderer(BaseAxisRenderer)**, setting **Name**, **Category**, **Description**, **RequiredRoles/OptionalRoles**, and optionally **Kwargs/Options**.
4. Implement **render_axis** using ordinary Matplotlib calls on **ax**.
5. Restart the application (or reopen the chart-creation window) — the new type appears in the picker with no further wiring.

app/charts/text.py is a good, compact, complete example to read or copy as a starting point: a handful of roles, a short **Kwargs** block, and a straightforward **render_axis**.

Writing a custom series operation

A series operation lives in **app/series_operations/**, as a class that subclasses **SeriesOperationDialogBase** (from **app.series_operations.dialog_base**) **directly** — discovered the same way, by **app/scanners/series_operation_scanner.py**.

The base class supplies the whole dialog shell (series picker, parameter form, preview pane, action buttons) and the plumbing that turns a result into a new series or table on the chart. A subclass sets a few class attributes —

Name	Display name shown in the Series Operations list.
Description	One-line summary shown next to the name.
Icon	An inline SVG source string for the operation's icon.

— and overrides the hooks the base class calls at the right moments. The ones every operation implements are:

build_parameter_selector()	Build the parameter form. Simple operations describe their parameters declaratively with the helpers in app.series_operations.parameter_spec (FloatParam , IntParam , ChoiceParam , ...) rather than laying out widgets by hand.
prepare_job() / finish_job()	Read the widgets into a job whose run() does the calculation without touching the window, and turn its outcome into results. With these two, Preview and OK work unchanged, and

	<code>evaluate(..., background=True)</code> runs the job on a worker thread with the shared Stop button — see the developer guide. An operation that does not need that can override <code>compute_results()</code> instead.
<code>result_series_spec(s)()</code>	The series metadata (name, roles, style) the base class writes to the chart. The DataFrame comes from the result's own <code>to_frame()</code> , and the table name from the <code>RESULT_TABLE_PREFIX</code> class attribute, unless you override them.
<code>format_results()</code>	Render the results as the text/HTML shown in the preview pane.
<code>refresh_results()</code>	Recompute and redraw the preview after a parameter changes.

Practical steps:

1. Create `app/series_operations/my_operation_dialog.py`.
2. Subclass `SeriesOperationDialogBase`, set `Name`, `Description`, `Icon`.
3. Declare parameters (e.g. with `FloatParam/IntParam/ChoiceParam`) and implement `build_parameter_selector`.
4. Implement `prepare_job/finish_job` (or `compute_results`), `result_series_spec`, and `format_results`. Put the arithmetic in a Qt-free module under `app/analysis/`, so it can be tested on its own.
5. Restart the application — the operation appears in the Series Operations list.

`app/series_operations/function_dialog.py` is the smallest complete dialog and a reasonable starting template; `app/series_operations/calculus_dialog.py` is a good second read for an operation that consumes an existing series rather than generating one from scratch.